

sessional - III

Page No.

Date :

sender's window = $\min(\text{Receiver's window}, \text{network's capacity})$

How Transport feel this?

→ Because segments are dropped & we not get ack

- MSS might be dependent on MTU, but not of them decide sender's window.

- Need to estimate network's capacity, while we know receiver's window exactly.

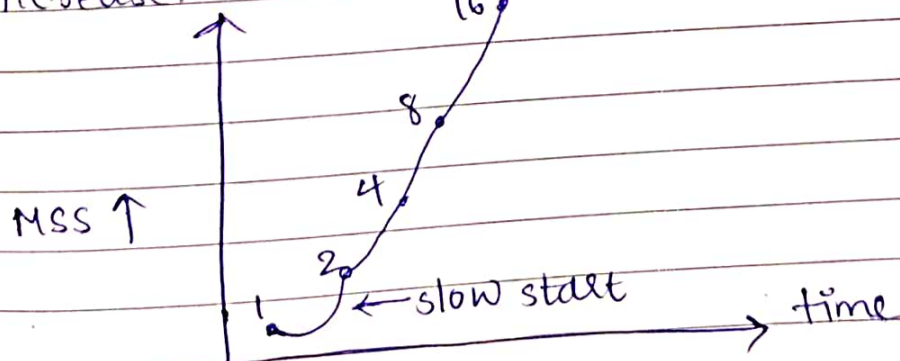
- transmitter's estimation on network capacity is "congestion window."

$$\text{sender's window} = \min(\text{Rx's window}, \text{congestion window})$$

→ How to find value of congestion window?

① Send size = MSS of Rx. if all data will be received then it is OK.
if some is dropped then, that size is congestion.

② send 1^{st} segment initially. and then slowly increase.



after sending
1 seg, after
getting ack
then only
seg. increase.

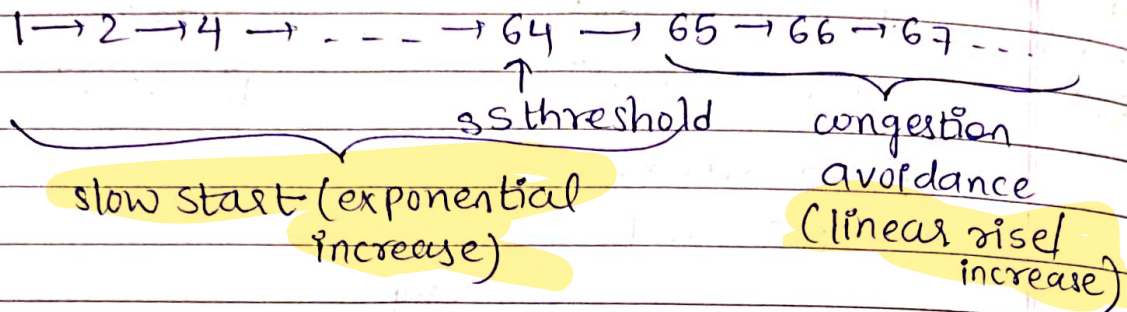
Q How a TCP sender can detect the possible existence of congestion in the network?

→ ① timeout event & ② receiving 3 dup. acks.

Page No.

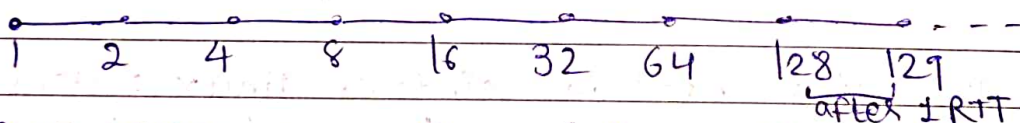
Date:

- double until "slow start threshold".



- whenever we have RTO at RX, there is congestion present.

- After RTO, $\text{ss threshold} = \frac{\text{ex (256)}}{\text{congestion window}} = 128$



⇒ After RTT, congestion window will be doubled if we are in "exp increase" or increase by 1 if we are in "linear rise"

⇒ On arrival of an ack

if (slow start)	} doubled eventually
$\text{cwnd} = \text{cwnd} + 1$	
if (linear rise)	} increased by 1.
$\text{cwnd} = \text{cwnd} + \frac{1}{\text{cwnd}}$	

→ At RTO, Reduce $\text{cwnd} = 1$.

→ Rather than reduce it to 1, → reduce it to half.
 $\text{ss threshold} = \text{cwnd} = \frac{\text{cwnd} + 1}{2}$ } SACK TCP or Reno TCP

old TCP

slow start and
then linear rise

Reno TCP

remain in
linear rise

($1 \rightarrow 2 \rightarrow 4 \rightarrow \dots$
 $128 \rightarrow 129$)

How sender knows that ^{why} ack is not coming
because of packet lost or congestion.

↳ NACK (Not used)

↳ ECN (if network is heavily congested
ECN itself might be dropped.)

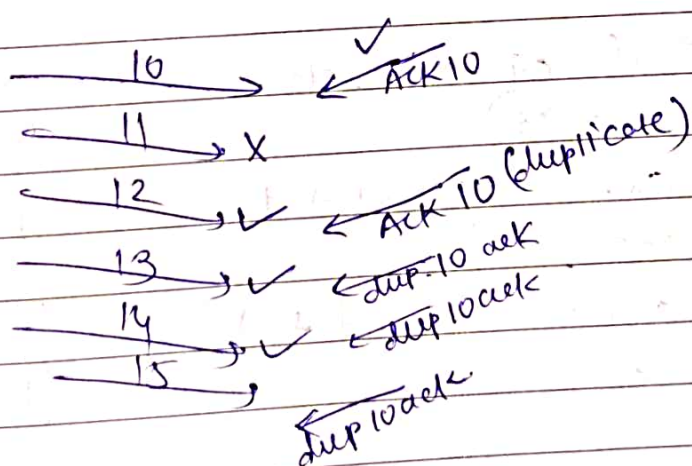
TCP version used in windows, linux systems:

Compound TCP

↳ Linux TCP

(CUBIC fin to
reducing window)

cwnd = 6



sender:
ACK 10
1st dup. ack 10
2nd dup. ack 10
3rd dup. ack 10.

Based on experiment
threshold = 3.
dup. packet

Fast Retransmission
of 11
takes place

why not 1? → 11 may come
after some time
so wait → not
Retransmit.

Why fast retransmission $\rightarrow$ to reduce time & improve efficiency.

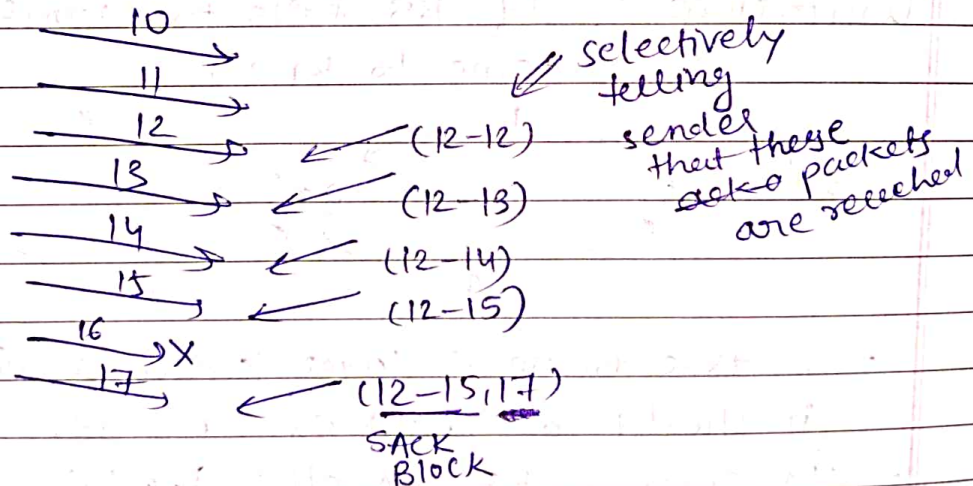
- There is used in "Tahoe TCP"

$\rightarrow$ Reno TCP

Along with fast retransmission also fast recovery instead of reducing $\leftarrow$
 $cwnd = 1$, $cwnd = ssthresh = \frac{cwnd}{2}$

$\rightarrow$ New Reno TCP : Modified Fast recovery.

$\rightarrow$ SACK TCP (Selective Ack TCP)

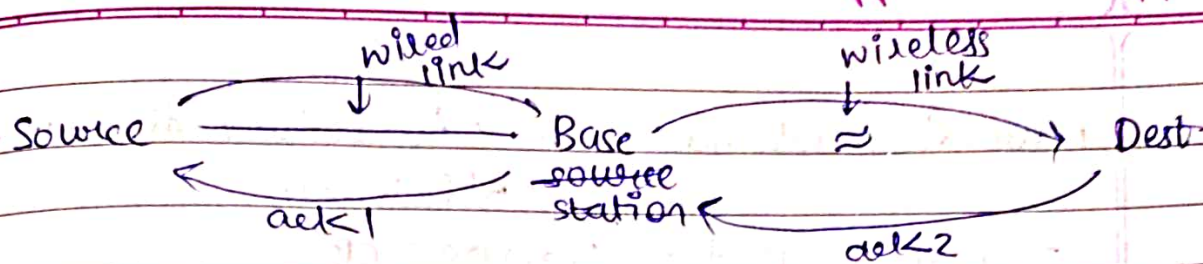


Q

How many SACK Blocks can be supported?

1 SACK $\rightarrow$ 1st seq no. - last seq.
 4 Bytes + 4 Bytes = 8 Bytes

total $\frac{40 \text{ Bytes}}{8} = 5$ But in options there are control bits (4 Bytes)
 $= \frac{32}{8} = \boxed{4}$



wired $\rightarrow$ lesser bandwidth congestion

wireless $\rightarrow$ higher bandwidth corruption

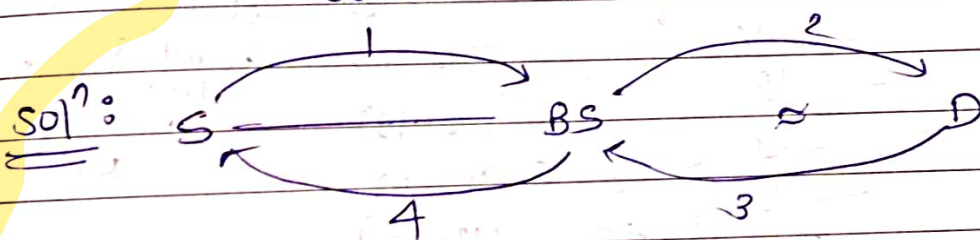
whenever ack1 is lost

source : RTO $\rightarrow$ reduce flow

(Bcoz of congestion) $\rightarrow$ Retransmission

dest : RTO $\rightarrow$ retransmit

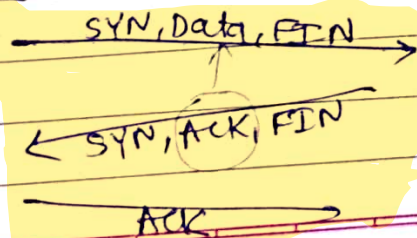
Limitations : If BS fails to transfer, S can never come to know.



- No true transport layer behaviour, violating behaviour it should be end-to-end, NO BS.
- It only works exactly like this (first wired & then wireless).

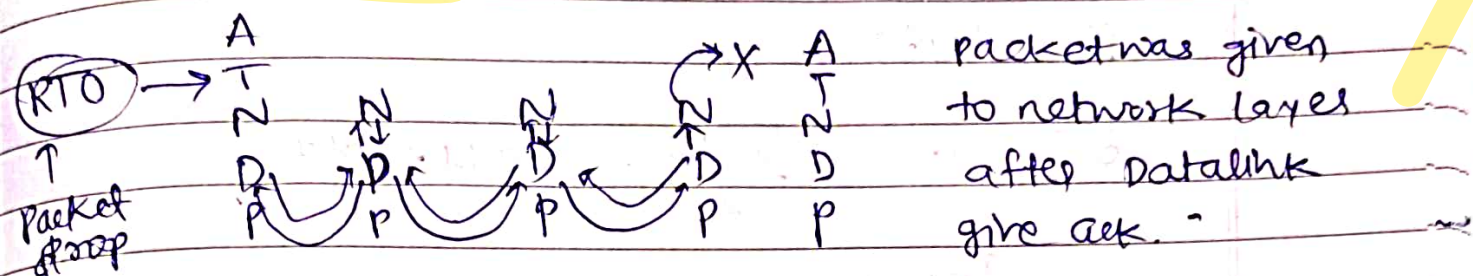
Transactional TCP

- Data is small but we want reliability then this is used.



Why datalink layer not helpful in congestion?

(source) A — R₁ — R₂ — R₃ — (destⁿ) B



Old TCP : loss based TCP.

Indirect TCP : loss based TCP and all till now are loss based TCP.

Compound = loss + Delay.

Delay based TCP : Westwood TCP (bandwidth estimation)

9 marks = TCP congestion + TCP variants. sess-III
3 marks = ARE + JAMP + ...

How sender works at Transport layer?

Takes data from appⁿ layer, make segment & Passes data to Network layer.

Job of Receiver TCP?

takes data from Network, verify checksum, if correct then send ack and send data to upper layer, if checksum is incorrect, discard data.

In which order ideally $\Rightarrow$ ack or write^{to} appⁿ layer

①

②

②

or

①

Crash Recovery:

receiver implementation

whenever
RX tells TX
he goes down
what TX does?

	Ack-write (AW)			Write-App ⁿ (WA)		
	Crash AW	AcW	AWC	CWA	WCA	WAC
Always Retransmit	(data is available only once) ok	ok	duplicate data.	ok	dup	dup
Never Retransmit	Data loss	loss	ok	loss	ok	ok
slightly better Retransmit if no ack	ok	loss	ok	ok	dup	ok
Retransmit if ack	loss	ok dup	dup	loss	ok	dup

⇒ If layer K crashes it never recovers from by itself
 $K+1$ might help K to recover from failure.

Comm. types : ① Peer-to-Peer ② Client-server

timers:

- ⇒ ① Retransmission timer: starts at sender after sending segment
- ② Persistent timer: start at sender when 0 window size is declared
- longest ⇒ ③ keep alive: (ex 20 min, 1 hour etc--)
 started on both side (Sender and Receiver)
 whenever there is activity it is stop. restart reset.
 If no activity, then it is terminated.
 (close the connection)

By closing connection we save.

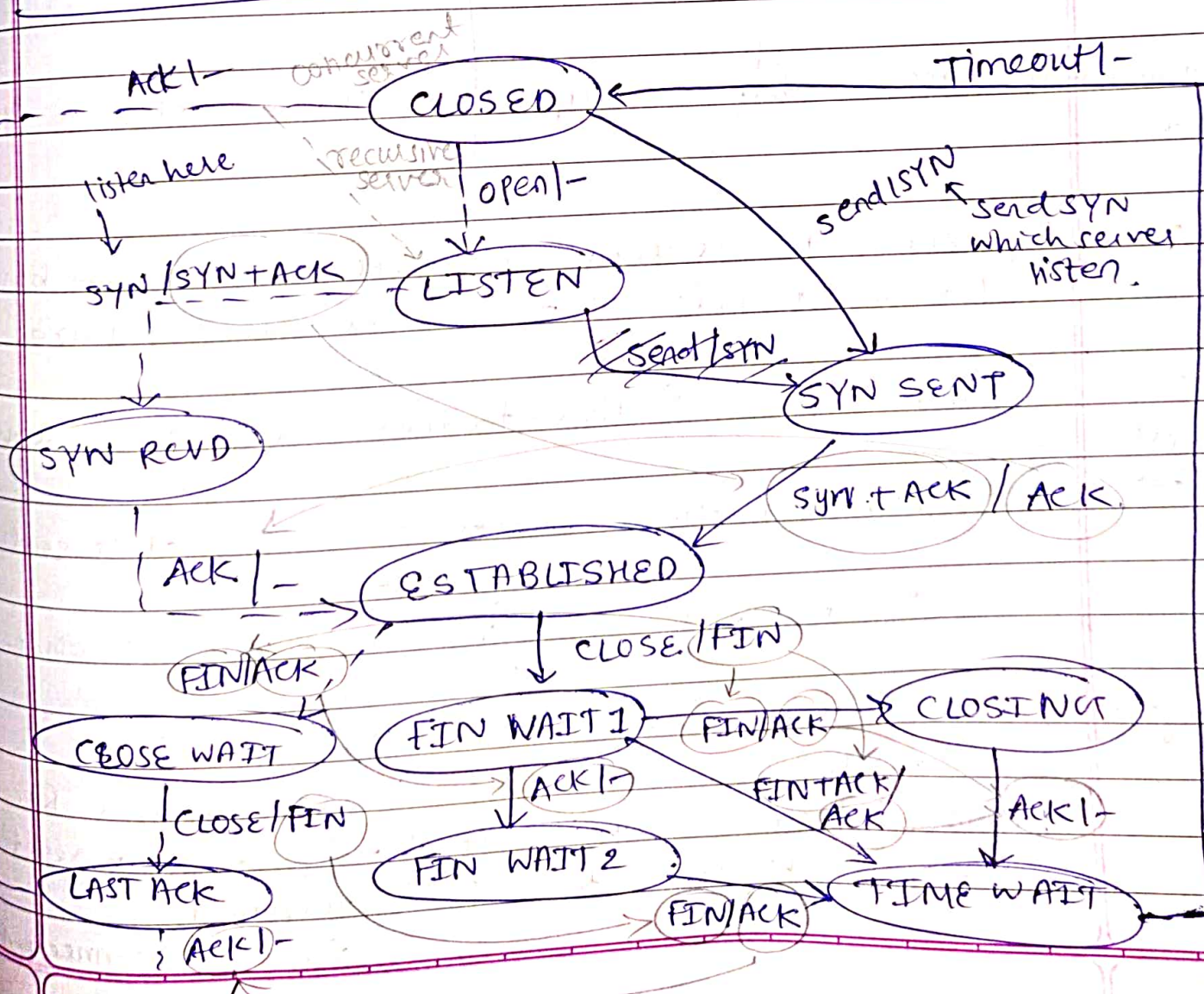
Peer-to-Peer: Port address, memory.

whoever initiates,
follow client
other, follow server

client-server: at server there is limit of how many clients it can entertain.
So, if there are "down client" it may not listen actual client.

TCP state Transition Diagram

How do we tune the Retransmission timer?
When timer expires cwnd will be decrease
and data rate become slow.



→ — : client - - - : server

event/respond.

→ In Time WAIT state - we start time wait times. whenever goes off, we close the conn.

↳ Duration : $2 \times \text{max. segment lifetime}$.

⇒ Why TIME OUT. ? (we wait twice).

①

→ FIN

← FIN+ACK

→ ACK

← If ACK of LAST ACK is dropped then if we not wait twice then server will keep changi sending FIN+ACK

② So that all the segments are not valid then only we close the connection.

all the seg.s are generated throughout the conn. If they are not valid then only we close the conn.

Self study